*Proceedings of the 2026 Winter Simulation Conference*
*V. Ramamohan, A. Djanatliev, M. Fakhimi, C. Krejci, C. Ruiz Martin, B. S. Onggo, and N. Mustafee, eds.*

# A TUTORIAL ON GENAI AND SIMULATION MODELING INTEGRATION

Mohammad Dehghani, Sahil Belsare, and Negar Sadeghi

Department of Mechanical and Industrial Engineering
Northeastern University
360 Huntington Ave, Boston, MA 02115, USA

## ABSTRACT

This tutorial provides insights into how GenAI and Large Language Models can support every stage of the simulation modelling workflow, from problem framing to evaluation. The aim is to clarify what these models do well in simulation today, what they still get wrong, and how to use them without giving up rigour. Building on the 2025 edition, this revision adds the Model Context Protocol as an agentic construction route, a heuristic versus LLM in the loop decision-rule comparison inside a running model, and a fully reproducible repository.

## 1 INTRODUCTION

Some of the most revolutionary products appear to be conceived in a single night, yet they are almost always built on decades of quiet, accumulating research. The simulation modelling community knows this story well. What looks today like a mature, multi-method discipline is the product of more than half a century of incremental progress, from the first simulation languages of the 1960s (GPSS, SIMSCRIPT, SLAM), through the GUI revolution of the 1990s (ProModel, Arena), to the multi-method platforms of the 2000s such as AnyLogic, FlexSim, and Simio (Figure 1).

Artificial intelligence has followed a strikingly parallel arc: rule-based and expert systems in the 1960s, classical machine learning in the 1990s, deep learning in the 2010s, and GenAI in the early 2020s.

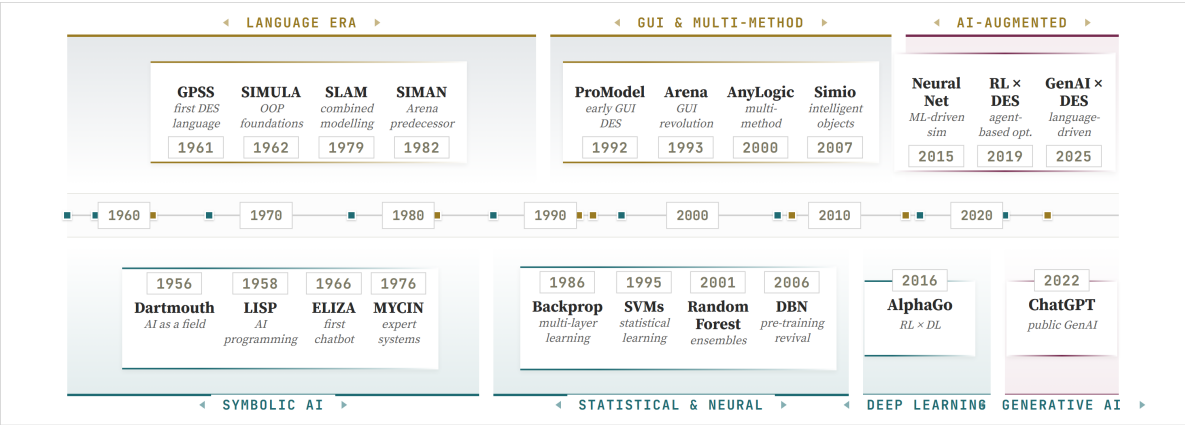



Figure 1: Notable milestones in both simulation modelling and artificial intelligence.

GenAI and Large Language Models (LLMs) have moved rapidly from research artefacts to general-purpose tools used across software, scientific, and creative work (M. Zao-Sanders 2025). The integration of LLMs with simulation modelling, however, did not gain the same momentum until recently. The shift came as frontier models grew capable of producing runnable code on technical problems (Chen et al. 2021; Rozière et al. 2024), at which point the same model that drafts a simulation model can also narrate its output and reason about its behaviour.

This paper continues the 2025 edition of the tutorial (Dehghani et al. 2025), which introduced a capability framework for LLM use across the discrete event simulation workflow. It is updated for a year in which GenAI has reshaped what is possible at a speed without precedent in computing history. Three shifts in particular motivate this revision. First, frontier models now produce simulation code that runs on the first attempt for cases that previously required iteration. Second, the Model Context Protocol (Anthropic 2024) has emerged as a standard way for language models to invoke external tools, opening up agentic construction patterns that were impractical a year ago. Third, the community has moved from conceptual tutorials toward reproducible artefacts that readers can run, fork, and extend.

This paper makes three contributions:

- A revised tutorial that restructures the 2025 framework around five workflow stages (Problem Formulation, Creation, Execution, Experimentation, and Evaluation), each containing three contributions of increasing LLM complexity.
- A working agentic AI workflow built on the Model Context Protocol that allows language models to construct simulation models by calling exposed simulation primitives, rather than generating monolithic source files.
- An empirical cross model comparison of two frontier large language models acting as decision agents inside a running simulation (bed assignment with the LLM in the loop), against deterministic heuristic baselines.

All artefacts that produced these results, including the SimPy model, prompt templates, evaluation harness, and a live demonstration in the browser, are released as a public companion repository.[1]

### 1.1 Running example: Emergency Department

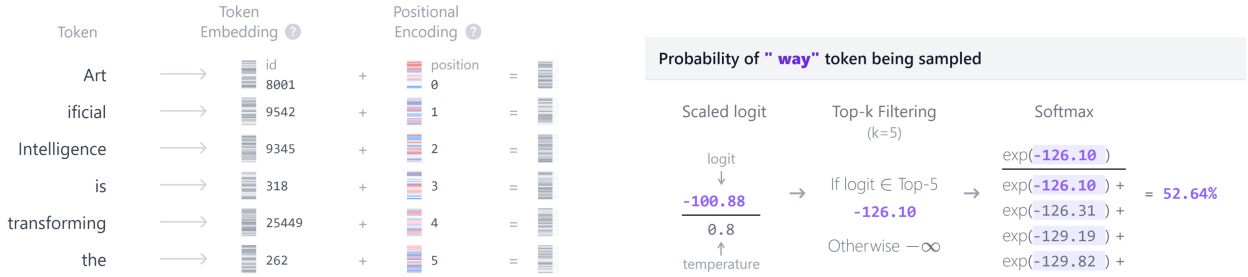
A multi-stage emergency department (ED) model serves as the single running case study throughout this paper. The purpose of using one consistent example is pedagogical, allowing us to showcase how the LLM contributes at each stage of the simulation workflow without continually switching contexts. The system models patients arriving at the ED, being triaged into severity bands, possibly passing through imaging or laboratory services, and ultimately being admitted to inpatient beds or discharged. The same case is later used to anchor two head-to-head contrasts in the paper, vibe coding against structured construction at the Creation stage, and a heuristic policy against an LLM-in-the-loop policy at the Execution stage.

## 2 LARGE LANGUAGE MODELS

GenAI has reshaped the technology landscape in the four years since ChatGPT's public release in late 2022, and at the centre of that change sits a single architecture, the large language model. An LLM predicts the next token in a sequence using the Transformer architecture (Vaswani et al. 2017), which replaced the strictly sequential processing of older recurrent networks with self-attention. Three steps run in order: the input string is tokenized and each token is mapped to a dense vector with a positional encoding (Figure 2a); a stack of Transformer blocks repeatedly applies multi-head self-attention and feedforward layers; and the final layer produces a probability distribution over the vocabulary, from which the next token is sampled (Figure 2b). Three parameters control how much variation that sample tolerates. *Temperature* rescales the distribution before sampling: low values concentrate mass on the most likely token, high values flatten the distribution (Figure 3). *Top-k* restricts the sampling pool to the $k$ highest-probability tokens. *Top-p*, also known as nucleus sampling, keeps the smallest set of top tokens whose cumulative probability exceeds a threshold $p$, which adapts to the local shape of the distribution. Throughout this paper we run with temperature 0.2 for code-generation prompts and 0.7 for narration prompts, with top-p 0.9 in both cases.

[1] Source and live demo: https://github.com/mdehghani86/wsc26-genai-sim and https://mdehghani86.github.io/wsc26-genai-sim/.

Our earlier tutorial (Dehghani et al. 2025) treats this material at full length and is the recommended primer for readers who want the lower-level view.




(a) Token embedding and positional encoding.

(b) Output probability distribution over the next token.

Figure 2: First and last steps of a Transformer pass.

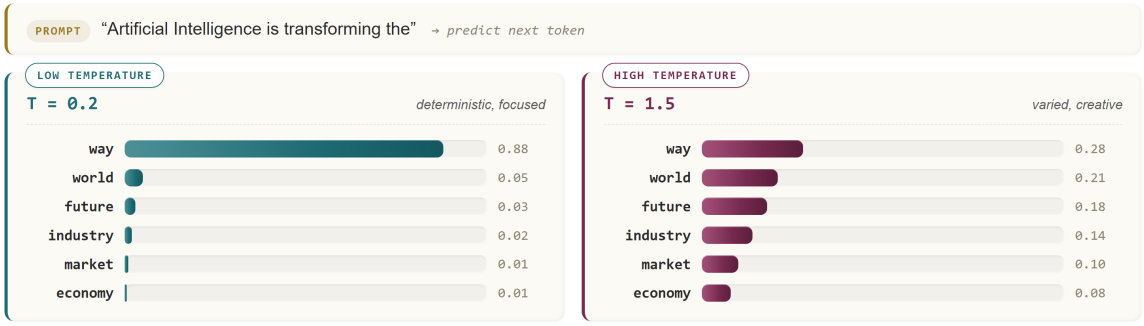



Figure 3: Same prompt under two temperatures: $T$=0.2 concentrates probability on a single token; $T$=1.5 flattens the distribution.

## 3 GENAI-DRIVEN SIMULATION FRAMEWORK

The paper organises GenAI-assisted simulation into five typed stages, each containing three rising-complexity LLM capabilities (Figure 4). The five stages trace the lifecycle of a simulation study: *Problem Formulation* captures what is being modelled, *Creation* turns the brief into runnable code, *Execution* runs the model and supports in-simulation decisions, *Experimentation* explores scenarios, and *Evaluation* closes the loop with reproducibility and benchmarking. A five-check validation gate at the end of Creation prevents broken models from propagating downstream.

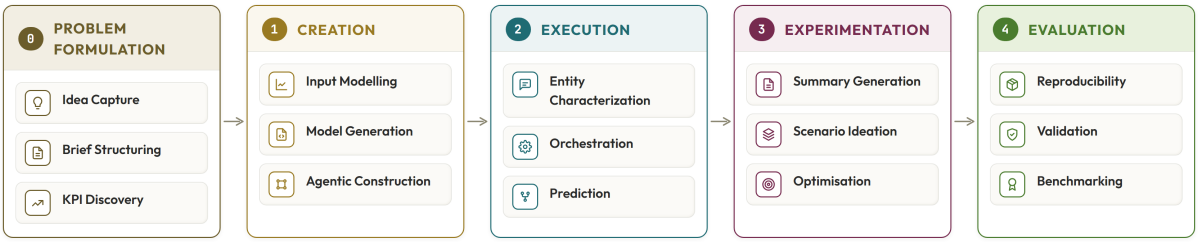



Figure 4: The GenAI-driven simulation framework with five stages, each contributing three rising-complexity LLM capabilities.

## 4 STAGE 0: PROBLEM FORMULATION

Problem formulation is the first and most consequential step of a simulation study. Without a clear definition of what is being modelled, what the scope is, and which metrics will judge success, the cleanest code and the most rigorous statistics still produce the wrong answer. This is where GenAI is already known to add the most value in professional practice: ideation, structuring, and surveying what others have done. The Problem Formulation stage applies these strengths to simulation through three rising-complexity LLM contributions:

- **Idea Capture.** The LLM is prompted in natural language with the study question and produces a candidate framing. This is the simplest contribution, a rephrasing rather than a verdict.
- **Brief Structuring.** The candidate framing is forced into a fixed three-row schema (problem, scope, KPIs) so that downstream stages consume the artefact verbatim.
- **KPI Discovery.** The LLM is asked to consult recent literature and propose KPIs that reflect community practice, rather than the modeller's prior habits, for the third row of the schema.

Recent analyses of professional GenAI adoption show that ideation, problem framing, and quick literature scans are among the most widely used everyday capabilities of these models (M. Zao-Sanders 2025). Simulation studies depend on exactly the same kinds of activities at the very start, where the modeller has to articulate the question, fix the boundary of the system, and choose the metrics by which the study will be judged. The Problem Formulation stage operationalises this overlap. The LLM is asked to fill a fixed three-row brief on problem, scope, and KPIs, and that brief is the artefact the rest of the paper carries forward unchanged. All prompts in this paper were issued to Anthropic's Claude[2] and reproduced verbatim from the chat transcript.

Figure 5 shows the prompt and response for the ED running example. The prompt does three things at once, stating the request in natural language, anchoring the model to the specific facility and study question, and asking the LLM to consult recent literature so that the returned KPIs reflect community practice rather than the modeller's prior habits. The response is constrained to a fixed three-row schema, and forcing that structure pays back in two practical ways. First, the output is directly comparable across projects and across language models. Second, it draws attention to whichever cell is weakest, almost always the scope-and-boundaries row, and lets that conversation happen up front rather than after a two-hundred-line script has already been written against an unstated assumption. The caveat that Si et al. raised about LLM ideation in general applies here (C. Si and D. Yang and T. Hashimoto 2024). Although the LLM recombines published practice rather than originating a new study design, it is the structured schema that transforms this recombination into a usable artefact downstream.

The three rows of the brief are not a deliverable in isolation. Each row drives directly into one of the three downstream stages of the workflow. The scope row determines which resources and processes appear in the model produced by Creation (Section 5), so any element it omits, ambulance routing for example, cannot reappear later in the simulation or in the narration. The problem-definition row sets the question that Execution (Section 6) must ultimately answer, since a trace that does not address it has run but produced no evidence. The KPI row supplies the success criteria of Experimentation (Section 7) and the targets against which the cross-model benchmark of Evaluation (Section 8) is judged.

## 5 STAGE 1: CREATION

Model creation is often the most time consuming step of any simulation study, since it requires the modeller to translate the system into runnable code while keeping every structural decision consistent. Commercial discrete event simulation packages are closed by design, which limits how directly external tools can be brought to bear on that step. Open Python ecosystems such as SimPy have grown over the past decade into

[2]Specifically, claude-opus-4-7. The empirical in-simulation comparison of Section 8 additionally exercises GPT-5.1 and Gemini-2.5-flash.

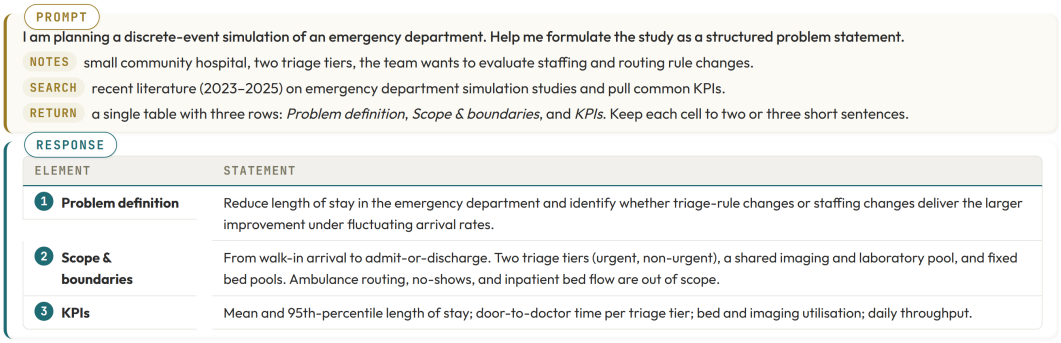



Figure 5: Problem-formulation prompt and response: a fixed three-row schema (problem, scope, KPIs) that downstream stages consume unchanged.

a usable target for general code generation tooling, so a frontier LLM that writes Python code can plausibly help with model construction. This motivates the vibe coding workflow used throughout this stage. The following subsections discuss how GenAI can support simulation model creation from three perspectives.

### 5.1 Input Modelling

Input modelling is a place where the general LLM strength (pattern recognition, plausible suggestion) and the general LLM weakness (no calibrated sense of statistical evidence) sit unusually close together. A frontier model will look at a histogram summary and confidently name a distribution that may or may not fit; what it cannot do is decide, on its own, whether the fit it proposed should be trusted. Recent work has explored two ways around this. SimLLM (J.-Q. Chen and K. Zhang and R. Zheng and Y. Zhong 2026) fine-tunes code LLMs specifically on SimPy queueing data so the model proposes simulation-aware distributions, and adjacent LLM-driven DES frameworks delegate post-run insight extraction to multi-agent pipelines (M. Selak and D. Krechel and A. Ulges and S. Spieckermann and N. Stoehr and A. Loehr 2026). Both approaches keep the language model as a central decision-maker in their respective loops.

We take a deliberately narrower position. Goodness-of-fit is a solved statistical problem, with distribution-free tests understood since Anderson and Darling (1952) and packaged in standard scientific computing libraries for two decades (Delignette-Muller and Dutang 2015). An LLM that picks a distribution by inspection is throwing away that work and adding a non-reproducible step in its place. The general principle is to use the LLM only at the two ends of the pipeline, proposing candidates and writing the verdict, and let standard goodness-of-fit tests have the final word in between. This keeps the LLM in its zone of competence (verbal proposal, verbal summary) and the statistical work in its proper place (mechanical and reproducible).

In practice the pipeline is short. A numeric column or timestamp series is sanity-checked and summarised, the LLM is asked for a handful of candidate continuous distributions with one rationale each and no fit attempted, the candidates are fitted programmatically, and the fits are assessed via Kolmogorov–Smirnov, Anderson–Darling, and a binned chi-square test against their fitted CDFs. The companion repository carries the exact code; the methodological point is that the LLM never sees the test result before it has finished proposing.

Figure 6 illustrates the failure mode this pipeline is designed to catch. On a deliberately bursty arrival trace, all three of the LLM's candidates are rejected at the 5% level. The empirical histogram and the LLM proposals look reasonable in isolation, but the Q-Q plot and the test column make the failure visible. The trace is non-stationary, mixing rush hours with off-peak periods, and no single marginal distribution can absorb that variation. The verdict at the bottom of the figure is what the LLM writes after reading the test output, and its recommendation, refit per hour-of-day, matches what classical input-modelling practice

would have suggested. The point is not that the LLM was wrong, but that the test is what made the failure visible.

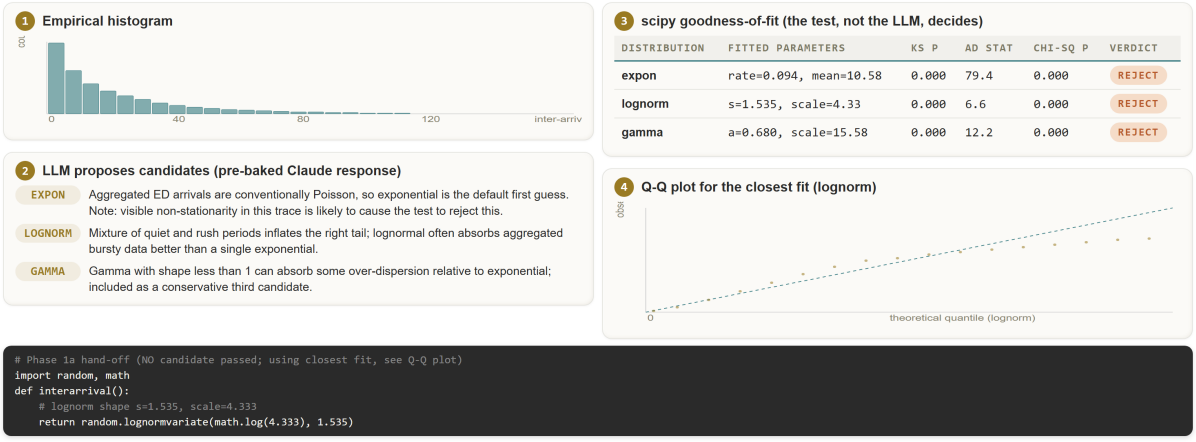



Figure 6: Input modelling on a bursty ED arrival trace. The LLM proposes three candidate distributions; `scipy.stats` runs KS, AD, and chi-square tests and rejects all three at the 5% level. The verdict recommends refitting per hour-of-day.

The artefact that leaves input modelling is a single random-variate generator together with the goodness-of-fit verdict that produced it. Model Generation (Section 5.2) consumes both, and a rejected fit travels with the artefact so the downstream prompt cannot quietly assume a global distribution it does not have. The connectivity is mechanical, not narrative: the brief's "arrival rate" row becomes a callable, and the verdict travels with it.

### 5.2 Model Generation

Simulation model creation requires the modeller to encode a long list of structural claims about a real system, including the capacity of each resource, the queueing discipline at each station, the routing logic between stations, the distributions that govern arrivals and service times, and the consistency of all of these choices with one another across the whole file. Each claim has to remain valid as the model evolves, because a single inconsistency can quietly invalidate every KPI the model produces. This density of interlocking decisions is what makes model creation the most demanding of GenAI's possible roles inside a simulation study.

Modern LLMs have shown the ability to write working code in unfamiliar libraries, follow long structured prompts, and pick up implicit conventions from a handful of examples, which makes simulation modelling a natural target for code generation. Open simulation ecosystems such as SimPy expose every primitive directly in Python, so the same kind of code that an LLM produces for a web endpoint can in principle express an M/G/c queue, an emergency department flow, or a small manufacturing line. Recent work supports this reading. SimLLM (J.-Q. Chen and K. Zhang and R. Zheng and Y. Zhong 2026) has shown that a fine-tuned 7B-class model can generate executable SimPy from a structured prompt template, and adjacent agentic DES work (M. Selak and D. Krechel and A. Ulges and S. Spieckermann and N. Stoehr and A. Loehr 2026) has shown that multi-agent LLM pipelines can carry meaningful simulation responsibilities elsewhere in the workflow. Together they make a credible case that frontier-model prompting is a defensible default for a tutorial reader.

The challenge is that a simulation file which compiles and runs is not necessarily a simulation file which is correct. Two generations from the same prompt can produce structurally different models, both

of which execute cleanly and both of which return plausible KPIs, while one of them silently violates an invariant the modeller never wrote down. The risk of silent failure is what shapes the rest of this subsection.

The case study that follows illustrates both the strengths and the limits of an LLM-driven model creation workflow on the running emergency department example. It shows the kinds of constraints and capacities the approach can express in its current form, the structural failure modes it is liable to introduce, and a set of natural extensions that would lower the failure rate further as the surrounding tooling matures. The two mechanisms used in this case study are a structured prompt that fixes the LLM's framing and reuses the artefacts from earlier stages, and an automated validation gate that catches the structural failures a human review would miss.

The structured prompt fixes the LLM's task framing and lets the same prompt be re-issued mechanically whenever the upstream brief changes. The template, introduced in our earlier tutorial, has five rows for role, goal, scenario, inputs, and output. The scenario row is copied from the Stage 0 brief verbatim, the inputs section reuses the random-variate generator from §5.1, and the rest is mechanical.

The automated validation gate is what actually does the work. Frontier LLMs are powerful but probabilistic: on the same prompt, two runs can produce structurally different simulation files, and silent bugs can survive a casual read. The companion implementation places a fixed five-check gate between the generated file and the downstream stages (Figure 7). Process logic checks that the entity flow matches the brief end to end. Connectivity checks that each producer wires into the declared consumer with no orphan resources. Parallel-versus-sequential checks that a pool with capacity $c$ actually serves $c$ entities in parallel; the canonical silent failure, identified in the WSC 2025 lessons-learned review of this case study, is a single per-pool "manager" process that collapses effective parallelism to one even when `Resource(capacity=c)` is declared correctly. Distribution checks verify that sampled service times match the input-modelling fit in mean, variance, and support. Theory checks compare realised utilisation, mean queue length, and mean waiting time against closed-form Erlang-C and Allen–Cunneen predictions for the matching $M/G/c$ system. A failed check is appended to the next prompt verbatim and the LLM regenerates only the affected section; after three rounds of failure, the case is handed off to the agentic-construction route of Section 5.3. The artefact that leaves Creation is therefore not just a simulation file, it is a simulation file plus a passing validation report. Either both travel forward, or neither does.

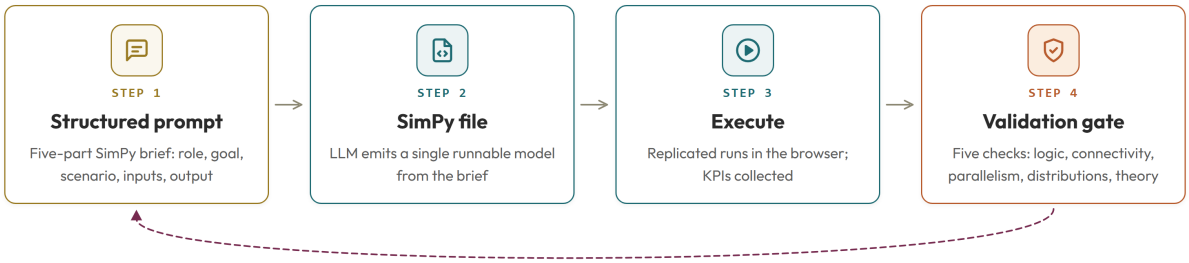



Figure 7: Vibe-coding loop: a structured prompt yields a SimPy file that is executed and screened by a five-check validation gate; any failing check is fed back into the next prompt.

The validated simulation file, with its passing report, is published in the companion repository (Section 8); Execution and Experimentation consume it without further modification.

### 5.3 Agentic Construction

Vibe coding behind a validation gate is one way to let an LLM author a simulation. It works at tutorial scale, but the interaction pattern has a built-in limit: the model emits a complete file in one shot, the gate passes or rejects it as a whole, and a rejection restarts the entire emission. Any failure, however local, becomes a full regeneration, and the diagnostic the model receives is the failed check rather than the offending line.

The general alternative is to constrain the LLM to construct the model through a small set of typed actions and let the surrounding tool layer enforce invariants on every call.

The Model Context Protocol is an open standard, introduced by Anthropic in November 2024, for connecting language models to external tools and data sources (Anthropic 2024). An MCP *server* exposes three primitives: tools (functions the LLM can call), resources (read-only data the LLM can request), and prompts (parameterised templates). An MCP *client*, embedded in the LLM host application, discovers the server's capabilities at session start and routes the model's tool calls to the server over JSON-RPC. The protocol has been the subject of two recent surveys, one focused on MCP itself (A. Singh and A. Ehtesham and S. Kumar and T. Talaei Khoei 2025) and one positioning it within the broader landscape of agent interoperability protocols (A. Ehtesham and A. Singh and G. K. Gupta and S. Kumar 2025).

The relevance to simulation is direct. A discrete-event simulation model is in essence a small set of resource declarations and a small set of process definitions. If those operations are exposed as MCP tools, the LLM no longer needs to write a complete code file in one shot. Instead, it issues typed calls to define resources, define processes, wire producers to consumers, run replications, and query KPIs, and the server enforces the relevant invariants on every call. Invalid arguments and unsatisfied dependencies are rejected at call time rather than after the entire file is regenerated, and theory checks running alongside the simulation produce diagnostics the LLM can read and act on without rewriting the whole model. The construction of the model becomes a structured dialogue with a typed API rather than a bet on a one-shot code emission.

What this buys an LLM in any DES setting is granular feedback. Construction proceeds as a sequence of typed calls: the model defines resources and processes, wires producers to consumers, asks for a run, queries KPIs. A wrong wire returns a typed error that the LLM corrects with one call rather than a full regeneration; a theory-check disagreement on utilisation points the LLM at the offending parameter rather than at the file as a whole. The companion repository ships a minimal simulation-MCP server with exactly this small set of primitives (resource definition, process definition, wiring, replication runs, KPI queries); the ED example is built end to end through a sequence of such calls (triage, imaging, lab, beds; arrivals, triage, admit or discharge).

The two routes are complementary rather than competing. Vibe coding with a validation gate suits a modeller who wants a single file to read, version, and run outside any agent loop; agentic construction suits incremental authoring, situations where invariants must be enforced at construction time rather than after the fact, and briefs that will be re-run against shifting parameter sets. Recent work confirms the trade-off from different angles: Vasa and Sarjoughian (2025) take the structured construction route via PDEVS-LLM with a statechart-driven workflow, and J. Kleiman and K. Frank and J. Voyles and S. Campagna (2025) present a general simulation-agent framework that decouples natural-language scenario configuration from model execution. Our contribution is the side-by-side comparison itself, with both routes shipped in the companion repository (Figure 4) and evaluated on dimensions agreed in advance: iteration count, first-pass executability, and diagnosability of failures.

## 6 STAGE 2: EXECUTION

During execution, a language model can move from author of the simulation to participant in it. What it can do during a run is different in kind from what it does at design time, and the simulation modeller now has a small set of new collaborators to think about rather than a single code writing assistant. *Entity Characterization* sits at the boundary where new entities arrive, when their attributes are still soft and a language model can give them richer descriptions than a numeric draw allows. *Orchestration* sits at the recurring decision points along their path, where the deterministic rule that has historically lived in code can give way to a policy that reads the live state and reasons about it. *Prediction* sits over the trace that the run has already produced, where reading the recent past and forming an expectation about what is coming next is a useful contribution in its own right, and one that can feed back into the previous two. The Execution stage takes these three in that order, from the lightest contribution to the most demanding.

Across the three contributions, a common pattern emerges. The language model is not asked to take over the simulation, only to step into specific moments where the modeller would otherwise have to encode something by hand or accept a fixed default. Each contribution is therefore judged by what it adds to the simulation rather than by how much of the simulation it replaces, and the case study that anchors each subsection is designed with that division of labour in mind.

### 6.1 Entity Characterization

Most simulation models initialise their entities with a fixed set of numeric attributes, an arrival time, a severity band, a service-time draw, chosen by the modeller at design time. Real systems carry far more information at the same moment of entity creation, and much of it is unstructured. LLMs are unusually well placed to convert that information into structured attributes that the rest of the model can act on. A patient checking into an emergency department arrives with a free-text complaint that an LLM can summarise into a triage severity, an estimated complexity, and a flag for whether imaging is likely to be needed. A customer arriving at a service centre can be tagged with intent and urgency from the wording of their request. A truck arriving at a port can be classified by cargo type from the booking note. The general capability is the same one that frontier models exhibit on classification and information-extraction tasks elsewhere, brought into the simulation at the point where attributes are first assigned.

The simplifying assumption that makes this contribution easy to deliver is that characterization happens once, at the moment of entity creation, rather than continuously during the run. The simulation does not need to call an LLM mid-run, and the only API calls happen offline as part of a pre-processing step that produces a static lookup keyed by entity. The companion repository ships such a lookup for the running emergency-department example, generated from a small set of synthetic patient backstories, and the simulation reads attributes from it as new patient entities are spawned. The methodological point of this subsection is that an LLM can move information that the modeller would otherwise have to encode by hand into a form the model can consume, without the latency, cost, or reproducibility issues that come with calling the LLM live.

### 6.2 Orchestration

The next capability up the ladder is orchestration: letting the LLM act as the decision maker at recurring choice points during the simulation. A discrete-event model typically contains a small number of such decisions (which server to assign to, which order quantity to release, which route to take), and each one is conventionally expressed as a deterministic rule of the current state. Letting a language model take that role replaces the rule with a context-aware policy that can change its reasoning across replications, but introduces three failure modes worth naming up front: per-decision latency on the order of hundreds of milliseconds, non-reproducibility unless temperature and prompts are pinned, and the absence of any automatic variance-reduction reasoning.

Recent work has demonstrated the pattern in adjacent simulation domains. Y. Quan and Z. Liu (2024) place an LLM agent inside a multi-echelon inventory simulation as the order-quantity decider, and Wang et al. (2025) place one inside a transportation simulation as the route-choice decider. In our tutorial case the recurring decision is bed assignment: each arriving patient must be placed in one of $b$ inpatient beds or held in queue. Three deterministic rules are natural baselines, round-robin, severity-priority with time-in-queue tie-break, and a clinician-style rule that balances bed turnover against remaining length of stay. The LLM-in-the-loop variant is added as a fourth option: at each assignment step the simulation pauses, serialises the relevant state, sends it to an LLM with a short instruction, and uses the returned index. The pseudocode is short enough to fit on a slide:

```
def llm_assign(state, llm):
    """Decide which bed index to use, given the current ED state."""
    prompt = render_prompt(state) # serialise occupancy, severity, queue
```

```
    reply = llm(prompt, temperature=0.0, max_tokens=8)
    return parse_bed_index(reply) # int in [0, b)

# Drop-in: the SimPy process calls llm_assign(state, llm) at each
# admission step instead of the round-robin or severity-priority rule.
```

What this pattern requires, in general, is a small amount of memory. A sequence of independent LLM calls is not a policy; it becomes one only when each call sees a compact record of what was decided before. The companion implementation does this with a rolling ring of recent decisions plus a one-line "policy so far" summary that the LLM compacts periodically; both are appended to the next prompt alongside the live state. Three operational properties then follow, and they are properties of the pattern, not of any particular study. First, latency matters: a frontier-model call adds hundreds of milliseconds to seconds of wall-time per decision, which adds up across $10^5$ assignments. Caching identical state vectors and batching calls in trace replay keeps the cost within tutorial budgets. Second, reproducibility matters: pinning temperature to zero and recording prompt-and-reply for every decision lets the run be replayed exactly. Third, the comparison is the point: running the same trace through every heuristic and through the LLM gives a row-by-row KPI table, judged against the same brief, with an agreement-rate diagnostic between LLM and the closest matching heuristic. The benchmark that follows operationalises this comparison on the ED case.

A small vendor-neutral driver keeps the pattern portable. The same loop runs with any frontier model behind it, the heuristics are exposed as built-in baselines, and the user supplies their own API key only for the LLM-in-the-loop variant. The methodological point is not UI-shaped: a simulation engine that can call out to a language model at decision time is a different artefact from one whose decision rule is fixed at compile time, and the methodological consequences (latency, cost, reproducibility, agreement-rate diagnostics) are what we care about.

**Case study: bed-assignment benchmark.** The case study below exercises this pattern across four admission policies on the same eight paired seeds: *FIFO* (oldest queued patient first), *severity-priority* (lowest severity number first, ties by arrival time), *GPT-5.1 with rolling memory*, and *Gemini-2.5-flash with rolling memory*. Each replication generates 25 patients across 4 beds with severity-stratified log-normal length-of-stay, producing 7–20 admission decisions per replication and 200 LLM calls per model in total. Both LLM policies see the live state, the queue, the last few decisions, and a periodically compacted "policy so far" summary; temperature is fixed to zero, so the only randomness across replications is the simulation seed.

Table 1: Admission-policy benchmark on the ED running example: $N$=8 paired replications, 25 patients each, B=4 beds. All wait times in minutes, mean $\pm$ std across replications.

| Policy | Mean wait | p95 wait | Sev-1 wait | Sev-3 wait | Util. | Decision lat. |
|---|---|---|---|---|---|---|
| FIFO | $28.2 \pm 21.2$ | $71.5 \pm 33.4$ | $27.9 \pm 23.5$ | $30.0 \pm 30.6$ | $0.83 \pm 0.08$ | $\sim 0$ ms |
| Severity-priority | $33.0 \pm 27.3$ | $112.3 \pm 79.0$ | $12.9 \pm 8.9$ | $64.1 \pm 76.5$ | $0.85 \pm 0.09$ | $\sim 0$ ms |
| GPT-5.1 + memory | $33.0 \pm 27.3$ | $112.3 \pm 79.0$ | $12.9 \pm 8.9$ | $64.1 \pm 76.5$ | $0.85 \pm 0.09$ | 773 ms |
| Gemini-2.5-flash + mem. | $31.7 \pm 24.7$ | $93.5 \pm 54.2$ | $12.9 \pm 9.0$ | $48.1 \pm 50.6$ | $0.85 \pm 0.09$ | 585 ms |

Three patterns deserve attention, and each one generalises beyond the ED case. First, FIFO and severity-priority differ along the expected axis: severity-priority more than halves urgent-patient wait (from 27.9 min to 12.9 min) at the cost of doubling routine-patient wait. The mean wait alone hides this trade-off, which is a generic property of any priority rule that pays no attention to the unpromoted class. Second, GPT-5.1 with memory reproduces severity-priority *exactly*: 0 of 200 decisions deviated from the rule, even though the prompt framed the trade-off rather than naming a target. The agreement-rate diagnostic from the orchestration pattern did its job: where an LLM agrees with a heuristic, the LLM is doing what

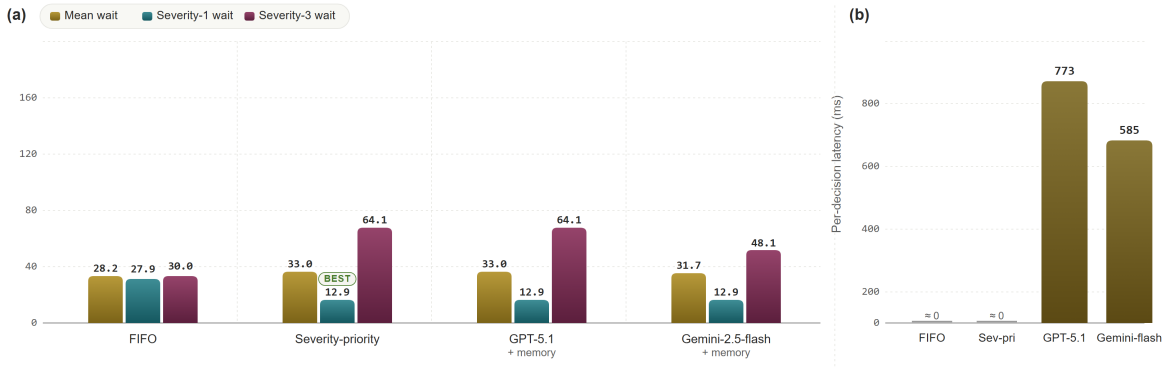



Figure 8: (a) Per-policy wait times; (b) wall-time cost per admission decision.

a sensible rule would do, at a wall-time cost (here about 770 ms per decision) the modeller can pay or refuse on its merits. Third, Gemini-2.5-flash with memory deviates on 16.5% of decisions (33 of 200), and the deviations are not random; they cluster on the longest replications, where queues lengthen and routine patients risk starvation. On replication 1, Gemini interleaves moderate and routine patients with the urgent ones and cuts routine wait from 218 to 105 minutes while keeping urgent wait essentially unchanged (18.1 min vs 18.5). This is the kind of contextual hybridisation a static rule cannot express, and it is the value proposition that justifies an LLM-in-the-loop variant in any simulation study where state-dependent trade-offs matter.

The cost of that value is non-trivial. A 600–800 ms decision latency adds up across $\sim 100$ admission decisions per simulated replication, and the LLM policies were $10^5$ times slower per decision than either heuristic. For a single tutorial-scale model this is acceptable. For a study with millions of decisions per scenario it is not. The companion repository ships a caching layer (identical state vectors return their prior decision) and a batched-replay mode for offline scenario sweeps; both reduce effective latency to the order of the heuristics. The methodological point of this case study, however, stands without those engineering details: a memory-aware LLM-in-the-loop is a genuinely different artefact from a fixed rule, and whether the difference is worth its cost is a property of the study, not of the modelling stack.

### 6.3 Prediction

The third execution capability is predictive. Instead of choosing what to do now, the LLM reads the recent simulation history and forecasts what is about to happen, so that downstream decisions can be informed by an expectation rather than by current queue state alone. The input to the prediction step is the recent `(time, entity, event)` log together with the brief, and the output is a structured forecast that the rest of the simulation can consume. Typical examples include an expected number of severity-1 arrivals in the next hour, a probability that the current ED census will breach a capacity threshold, and an estimated completion time for an order already in process. The general capability is the same one that frontier models exhibit on event-log and time-series tasks elsewhere, reasoning about a short past window and emitting a probable next-step description, brought into a simulation context where the past window is generated by the run itself.

Predictions of this kind are most useful when they feed back into orchestration. The bed-assignment decision becomes materially better when it can weigh the cost of placing a patient against a one-hour forecast of incoming arrivals, since admitting a routine patient now may shut down a bed that an urgent patient will need shortly. The same logic applies to staffing decisions, to inventory reorder triggers, and to any choice whose right answer changes when the next event is anticipated rather than reacted to. Prediction also generalises the older idea of post-run log analysis, where the LLM reads a finished trace and writes a

summary of what happened, by moving that same kind of reading inside the run and turning the summary into an input for the next decision. The companion repository carries a small log-summarisation utility that demonstrates the pattern on the running case study: a rolling window of recent admission events is summarised, passed to an LLM with a short forecast prompt, and the resulting prediction is logged alongside the deterministic queue state. Wiring that prediction directly into the bed-assignment decision is a natural next step, and one that the headline experiments do not yet take, since doing so collapses prediction and decision into a single latency budget. The methodological point of this subsection is that the capability is straightforward to demonstrate once the log infrastructure exists, and that it is the natural next layer of complexity above an orchestration policy that reads only the live state.

## 7 STAGE 3: EXPERIMENTATION

Once the model runs, the modeller has to design scenarios, sweep parameters, and turn the resulting numbers into something a reader can act on. Designing the scenario set, choosing which figures to show, and searching for good operating points are all places where an LLM can shorten the cycle without taking over the methodological choices. The Experimentation stage exposes three rising-complexity LLM contributions: *Summary Generation*, figure selection plus a short written interpretation of results; *Scenario Ideation*, proposing factors and levels from the brief; and *Optimisation*, using the LLM to drive parameter search, with explicit caveats.

### 7.1 Summary Generation

The most reliable Experimentation contribution from an LLM combines figure selection with narrative interpretation. Drawing plots from a log file is well-handled by deterministic code, so what the LLM is asked to do here is choose which two or three plots best support the comparison the brief calls for, given the KPI list and the scenario table. Because the plots themselves are produced by code rather than by the LLM, the figures stay reproducible from the log file alone; the companion repository ships the plotting utility.

The narrative half plays directly to a general LLM strength: writing a short interpretive summary of a numeric table for a human reader. The LLM is given the KPI rows from the brief, the scenario table, and short captions for the plots, and is asked to write a paragraph addressed to a stakeholder, naming which scenario performs best on which KPI and where the trade-offs lie. The output is reviewed by the modeller before it leaves the loop. The reason for keeping a human in this seat is the same as in input modelling: the model is good at recombining what it sees, but the responsibility for the claim rests with the modeller. Recent work on LLM-assisted result interpretation in adjacent domains supports this division of labour, with the model handling first-draft narrative and the analyst keeping the final word (M. Selak and D. Krechel and A. Ulges and S. Spieckermann and N. Stoehr and A. Loehr 2026; J. Kleiman and K. Frank and J. Voyles and S. Campagna 2025).

### 7.2 Scenario Ideation

The factors and levels are not invented from scratch. They are read off the brief. The KPI row tells us what to measure (e.g., mean length of stay, 95th-percentile waiting time, bed-utilisation). The scope row tells us what is movable in the system under study (e.g., bed count, triage staffing, hour-of-day arrival pattern). A small full-factorial or fractional-factorial design over the movable factors, replicated to the precision the KPI row demands, produces the data table. The LLM contribution at this stage is bounded and explicit. It proposes a starting design from the brief, the modeller accepts or trims it, and the rest is mechanical: generate the scenario file, run, collect.

### 7.3 Optimisation

Optimisation is the most complex Experimentation capability we attempt with an LLM in the loop, and the one we have the least to claim. A frontier LLM can suggest the next scenario to run given results so far — a loose Bayesian sequential design pattern that converges if the response surface is well behaved. It is not, however, a substitute for a real simulation optimiser such as OptQuest, nor for a black-box optimiser like SMAC. Two failure modes are worth naming. The model has no automatic exploration-versus-exploitation budget; without an explicit instruction, it tends to exploit. And it has no systematic way to reason about variance reduction; it does not know that a doubled replication count quarters the standard error. We therefore use the LLM-driven optimiser only on small designs with cheap evaluations, and report the search trace alongside the optimum so the reader can judge.

## 8 STAGE 4: EVALUATION

A simulation study is only as useful as the trust the reader can place in its numbers. The Evaluation stage closes the workflow by re-running everything against reference instruments and by exposing the same harness to multiple frontier LLMs, so that the cross-model claims rest on a uniform measurement rather than a single chat session. Three rising-complexity LLM contributions sit at this stage:

- **Reproducibility.** Every figure, every prompt, and every result is regenerable from the public repository, and the live in-browser site lets a reader exercise the workflow without installing anything.
- **Validation.** Realised KPIs are cross-checked against closed-form $M/G/c$ bounds and against an off-the-shelf commercial reference model (Arena or Simio configured to the same brief), to provide a second-source check that does not depend on the SimPy implementation.
- **Benchmarking.** The general practice of running multiple frontier LLMs through the same harness on the same paired seeds, generalised as a reusable evaluation principle and applied to the admission-policy comparison in §6.2.

**Reproducibility.** Every artefact in this paper is reproducible from the public companion repository, which hosts the validated SimPy model, the input-modelling demos (clean and bursty), the prompts used at each stage, the validation harness, the tool-server used for agentic construction, and the comparison-table run logs. A companion live site lets a reader exercise the workflow without installing anything; the input-modelling pipeline runs end to end in the browser, and the LLM-in-the-loop variants are exposed under a user-supplied API key.

**Validation.** KPI claims throughout the paper are cross-checked against analytical $M/G/c$ bounds (Erlang-C and Allen–Cunneen for the matching multi-server model), using the same validation harness invoked at each stage of the workflow. As a second-source check we additionally reproduce the brief in an off-the-shelf commercial package (Arena or Simio) configured to the same arrival distribution, service-time fits, and resource pools, and report the realised KPIs side by side with the SimPy run.

**Benchmarking.** The Evaluation step that makes vendor comparisons trustworthy is to run every LLM under examination through the same harness on the same paired seeds, so that any difference in the resulting numbers reflects the model rather than the test bed. The bed-assignment results reported in §6.2 are a worked instance of this principle, where four admission policies (two heuristic, two LLM-in-the-loop) share a single decision driver and a single seed list. The general practice for a tutorial reader is to fix every part of the run that is not the model under test, to pin temperature to zero, to record every prompt and reply for replay, and to report mean and spread across enough paired seeds for the comparison to survive a sceptical review. The same harness extends to any execution-stage capability the modeller wants to benchmark across vendors, with the orchestration case study above as the template.

**Discussion.** This tutorial has organised GenAI-assisted simulation work into a 5 by 3 capability ladder with typed hand-offs and a validation gate, and has used a single emergency-department case study

to walk it end to end. The argument throughout has been that LLMs are best deployed at the boundaries of each stage, where they propose, narrate, and synthesise, and that the substantive decisions inside each stage are best left to instruments that already exist for that purpose. Goodness-of-fit tests pick the input distribution. Closed-form Erlang-C bounds judge the model. Mechanical sweeps generate the scenarios. In each case the model is the writer, not the arbiter. Three concrete advances over the 2025 edition stand out. The first is reproducibility: every figure, every prompt, and every result in this paper is regenerable from the public repository, and the live in-browser site lets a reader exercise the workflow without installing anything. The second is the agentic construction route, which replaces one-shot vibe coding with structured construction through a typed tool catalogue and is shown side by side with vibe coding on the same brief. The third is measurement: the same validation gate that protects the workflow also produces a uniform scorecard for comparing frontier LLMs, and the pilot benchmark reported above gives a first set of numbers on the ED case. None of these is a finished result, and the companion repository is structured so that readers can extend the comparison to their own briefs and their own models.

## REFERENCES


Anderson, T. W., and D. A. Darling. 1952. "Asymptotic Theory of Certain "Goodness of Fit" Criteria Based on Stochastic Processes". *Annals of Mathematical Statistics* 23(2):193–212.

Anthropic 2024, November. "Introducing the Model Context Protocol". Anthropic Engineering Blog. Accessed April 2026.

J.-Q. Chen and K. Zhang and R. Zheng and Y. Zhong 2026. "SimLLM: Fine-Tuning Code LLMs for SimPy-Based Queueing System Simulation". arXiv preprint arXiv:2601.06543.

Chen, M., J. Tworek, H. Jun, Q. Yuan, H. Ponde de Oliveira Pinto, J. Kaplan, *et al*. 2021. "Evaluating Large Language Models Trained on Code". *arXiv preprint arXiv:2107.03374*.

Dehghani, M., S. Belsare, and N. Sadeghi. 2025. "A Tutorial on Generative AI and Simulation Modeling Integration". In *Proceedings of the 2025 Winter Simulation Conference*, 1197–1211. Seattle, WA.

Delignette-Muller, M.-L., and C. Dutang. 2015. "`fitdistrplus`: An R Package for Fitting Distributions". *Journal of Statistical Software* 64(4):1–34.

A. Ehtesham and A. Singh and G. K. Gupta and S. Kumar 2025. "A Survey of Agent Interoperability Protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP)". arXiv preprint arXiv:2505.02279.

J. Kleiman and K. Frank and J. Voyles and S. Campagna 2025. "Simulation Agent: A Framework for Integrating Simulation and Large Language Models for Enhanced Decision-Making". arXiv preprint arXiv:2505.13761.

Y. Quan and Z. Liu 2024. "InvAgent: A Large Language Model based Multi-Agent System for Inventory Management in Supply Chains". arXiv preprint arXiv:2407.11384.

Rozière, B., J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan *et al*. 2024. "Code Llama: Open Foundation Models for Code". *arXiv preprint arXiv:2308.12950*.

M. Selak and D. Krechel and A. Ulges and S. Spieckermann and N. Stoehr and A. Loehr 2026. "Agentic Insight Generation in VSM Simulations". arXiv preprint arXiv:2604.12421.

C. Si and D. Yang and T. Hashimoto 2024. "Can LLMs Generate Novel Research Ideas? A Large-Scale Human Study with 100+ NLP Researchers". arXiv preprint arXiv:2409.04109.

A. Singh and A. Ehtesham and S. Kumar and T. Talaei Khoei 2025. "A Survey of the Model Context Protocol (MCP): Standardizing Context to Enhance Large Language Models". Preprints.org 202504.0245.

Vasa, V. K. S., and H. S. Sarjoughian. 2025. "Generative Statecharts-Driven PDEVS Modeling". In *Proceedings of the 2025 Winter Simulation Conference*. Code at https://github.com/comses/PDLM.

Vaswani, A., N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, *et al*. 2017. "Attention Is All You Need". In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 5998–6008.

Wang, L., P. Duan, Z. He, C. Lyu, X. Chen, N. Zheng, *et al*. 2025. "Agentic Large Language Models for Day-to-Day Route Choices". *Transportation Research Part C: Emerging Technologies* 180:105307.

M. Zao-Sanders 2025, April. "How People Are Really Using GenAI in 2025". Harvard Business Review. Accessed April 2026.

## AUTHOR BIOGRAPHIES

**MOHAMMAD DEHGHANIMOHAMMADABADI** is an Associate Teaching Professor in the Department of Mechanical and Industrial Engineering at Northeastern University. His research focuses on developing intelligent simulation frameworks integrated with optimization and AI-based models to support decision-making, with applications in healthcare, manufacturing, and supply chain. His email address is m.dehghani@northeastern.edu.

**SAHIL BELSARE** is a simulation scientist with expertise in digital twin technologies and simulation modeling. He applies machine learning models to enhance decision-making in complex systems, focusing on optimization and reinforcement learning integrated with simulation platforms. His email is belsare.s@northeastern.edu.

**NEGAR SADEGHI** is a Ph.D. student in Industrial Engineering at Northeastern University. Her research focuses on developing intelligent simulation models for pharmaceutical supply chains and production-distribution systems. Her email address is sadeghi.ne@northeastern.edu.